api-gateway — Teknik Genel Bakış

Proje	Servis	Dil / Runtime	Son Güncelleme
Tresaurio Kripto Portföy Yönetim Platformu	api-gateway	Go 1.26.1	2026-04-24

1. Servisin Amacı

api-gateway, Tresaurio platformunun tüm client isteklerini karşılayan tek giriş noktasıdır. Gelen HTTP ve WebSocket isteklerini downstream servislere yönlendirir; kimlik doğrulama, hız sınırlama ve hata yönetimi gibi cross-cutting concern'leri merkezi olarak uygular.

Downstream servisler bu gateway'i görmeden doğrudan dış dünyaya açılmaz; tüm trafik buradan geçer.

Sorumluluk	Açıklama
Reverse Proxy	İstekleri market-data ve exchange-service'e yönlendirir
JWT Doğrulama	Korumalı endpoint'lerde token doğrular, `X-User-ID` header'ını downstream'e iletir
Rate Limiting	IP bazlı, endpoint grubuna göre ayrı limitler — Grup A: 30 RPS, Grup B: 5 RPS, Grup C: 2 RPS
Circuit Breaker	Downstream servis art arda hata verirse devreyi açar, 503 döner — threshold'lar env var'dan servis bazlı yapılandırılabilir
Upstream Timeout	Her route grubu için configurable deadline — market-data 5s, exchange 10s; süre aşılınca 504 döner, goroutine serbest bırakılır
Health Aggregation	Tüm downstream servislerin durumunu tek endpoint'ten sunar
WebSocket Proxy	WebSocket bağlantılarını upstream servise tüneller
Correlation ID	Her request'e UUID üretir, tüm downstream çağrılarına `X-Request-ID` header'ı olarak iletir
Swagger UI	Tüm API endpoint'lerini tarayıcıdan interaktif olarak belgeler ve test imkânı sunar

2. Mimari

```
Client (HTTP / WebSocket)
        │
        ▼
┌──────────────────────────────────────────────────┐
│              api-gateway (port 9000)              │
│                                                   │
│  ┌──────────────────┐  ┌──────────┐  ┌──────────┐ │
```

```
│  │  Rate Limiter    │  │ JWT Auth  │  │  Logger /  │ │
│  │ A:30 B:5 C:2 RPS │  │(protected │  │  Recoverer │ │
│  │  (per-IP/group)  │  │  routes)  │  │            │ │
│  └──────────┬───────┘  └─────┬─────┘  └────────────┘ │
│          │               │                      │
│  ┌───────▼───────────────▼──────────────────┐ │
│  │              chi Router                   │ │
│  │                                           │ │
│  │  /health           → Health Handler       │ │
│  │  /api/v1/quotes/*  → CircuitBreaker       │ │
│  │  /ws/quotes/*      → WebSocket Proxy      │ │
│  │  /positions/*      → CircuitBreaker + JWT │ │
│  │  /api/v1/pnl/*     → CircuitBreaker + JWT │ │
│  │  /api/v1/orders/*  → CircuitBreaker + JWT │ │
│  │  /ws/positions/*   → WebSocket Proxy + JWT │ │
│  └──────────────────────────────────────────┘ │
└──────────────────────────────────────────────┘
           │                      │
           ▼                      ▼
   ┌──────────────┐     ┌──────────────────┐
   │ market-data  │     │ exchange-service │
   │ (ngrok URL)  │     │ (ngrok URL)      │
   └──────────────┘     └──────────────────┘
```

Middleware Zinciri

Her istek aşağıdaki sırayla middleware'den geçer:

Global (tüm isteklere uygulanır):

```
Request
   │
   ├─► CORS            (Access-Control-Allow-Origin: ALLOWED_ORIGIN env var;
Allow-Credentials: true; OPTIONS preflight → 204)
   ├─► Logger          (chi — tüm istekleri loglar)
   ├─► Recoverer       (chi — panic'leri yakalar, 500 döner)
   ├─► RequestID       (UUID üretir / X-Request-ID header'ını okur, context'e
yazar, response'a ekler)
   ├─► Metrics         (Prometheus sayaçları ve histogram)
   └─► RequestLogger   (method, path, status, latency, IP, userID, request_id)
```

Route Group (sadece ilgili gruba uygulanır):

```
    ├─► [RateLimiter]   (Grup A / B / C — IP başına bağımsız limitler)
    ├─► [JWTAuth]       (sadece korumalı route group'larında)
    ├─► [Timeout]       (market-data: 5s, exchange: 10s, WebSocket: yok)
    ├─► CircuitBreaker  (market-data veya exchange CB'si)
    └─► Proxy Handler   (reverse proxy / websocket proxy — X-Request-ID
downstream'e iletilir)
```

---

3. Bileşen Detayları

3.1 Reverse Proxy (`internal/handler/proxy.go`)

Go standart kütüphanesindeki `httputil.ReverseProxy` kullanılarak implement edilmiştir. Her downstream servis için ayrı bir proxy instance'ı oluşturulur.

```go
func NewProxy(target string) http.Handler {
    u, _ := url.Parse(target)
    proxy := httputil.NewSingleHostReverseProxy(u)
    proxy.Director = func(req *http.Request) {
        req.URL.Scheme = u.Scheme
        req.URL.Host   = u.Host
        req.Host       = u.Host
        req.Header.Set("ngrok-skip-browser-warning", "true")
        if id, ok := req.Context().Value(middleware.RequestIDKey).(string); ok &&
id != "" {
            req.Header.Set("X-Request-ID", id)
        }
    }
    proxy.ErrorHandler = func(w http.ResponseWriter, r *http.Request, err error) {
        if errors.Is(err, context.DeadlineExceeded) ||
errors.Is(r.Context().Err(), context.DeadlineExceeded) {
            http.Error(w, "upstream timeout", http.StatusGatewayTimeout)
            return
        }
        http.Error(w, "bad gateway", http.StatusBadGateway)
    }
    return proxy
}
```

`Director` fonksiyonu her istekte çalışır ve hedef URL'yi set eder. `ngrok-skip-browser-warning` header'ı ngrok uyarı sayfasını engeller. `X-Request-ID` context'ten okunarak downstream servise iletilir — böylece her servis aynı correlation ID'yi loglarına yazabilir.

`ErrorHandler`, `TimeoutMiddleware` tarafından iptal edilen context'leri yakalar: `context.DeadlineExceeded` hatası geldiğinde `504 Gateway Timeout` döner. Diğer upstream hataları (bağlantı reddedilmesi, vs.) `502 Bad Gateway` döner.

3.2 Upstream Timeout (`internal/middleware/timeout.go`)

Her route grubuna `r.Use(apimiddleware.TimeoutMiddleware(d))` ile uygulanır. Middleware, `context.WithTimeout` ile request context'ine bir deadline ekler ve `defer cancel()` ile goroutine'i her koşulda serbest bırakır.

```go
func TimeoutMiddleware(timeout time.Duration) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            if timeout == 0 {
                next.ServeHTTP(w, r)
                return
            }
            ctx, cancel := context.WithTimeout(r.Context(), timeout)
            defer cancel()
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}
```

Route Grubu	Timeout	Sebep
market-data (Grup A, B, C)	**5s** (env: `TIMEOUT_MARKET_DATA`)	Hızlı piyasa verisi; Etherscan gibi yavaş upstream'lerin goroutine'i bloklaması engellenir
exchange (auth + protected)	**10s** (env: `TIMEOUT_EXCHANGE`)	İşlem servisi biraz daha toleranslı olmalı
WebSocket (`/ws/*`, `/api/v1/ws*`)	**timeout yok** (`0` geçilir)	WS kalıcı bağlantıdır; deadline uygulanamaz

Timeout aşıldığında `httputil.ReverseProxy`'nin `ErrorHandler`'ı `context.DeadlineExceeded` hatasını yakalar ve `504 Gateway Timeout` döner. Goroutine `defer cancel()` sayesinde derhal serbest kalır.

3.3 JWT Doğrulama (`internal/middleware/auth.go`)

Korumalı route group'larına uygulanan middleware. HMAC-SHA algoritmasıyla imzalanmış JWT token'larını doğrular.

```go
token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (interface{}, error) {
    if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
        return nil, jwt.ErrSignatureInvalid
    }
    return []byte(secret), nil
})

// Doğrulama başarılıysa:
userIDFloat, _ := claims["user_id"].(float64)
```

```
userID := fmt.Sprintf("%d", int64(userIDFloat))
r.Header.Set("X-User-ID", userID)
```

Adım	Açıklama
Header kontrolü	`Authorization: Bearer <token>` formatı (REST için)
Query param fallback	`?token=<token>` — tarayıcı WebSocket handshake'te header gönderemediğinden JWT bu yolla da alınabilir
İmza doğrulama	Yalnızca HMAC yöntemi kabul edilir, diğerleri reddedilir
Claim çıkarma	Token içindeki `user_id` claim'i (float64) alınır, int64'e çevrilir
Header iletimi	`X-User-ID` olarak downstream servise iletilir
Context	`userID` değeri request context'ine de yazılır

Hatalı veya eksik token durumunda `401 Unauthorized` + `{"error":"unauthorized"}` döner.

> **Not:** `JWT_SECRET` env variable boş bırakılırsa `auth.go` dev ortamı için `"default-secret-change-in-production"` kullanır. Production'da mutlaka set edilmeli.

3.4 Rate Limiter (`internal/middleware/ratelimit.go`)

Redis tabanlı sliding window algoritması ile implement edilmiştir. Tüm pod'lar aynı Redis instance'ını paylaşır; sayaç merkezi olarak tutulur. Gateway yeniden başlatıldığında ya da birden fazla instance çalıştırıldığında kota korunur.

Tek global limit yerine endpoint grubuna göre bağımsız limitler uygulanır. Her grup kendi chi `r.Group()` bloğunda tanımlanır; middleware sadece o gruba ait route'lara uygulanır.

Endpoint Grupları

Grup	Endpoint'ler	Limit	Redis key
A	`/api/v1/quotes/*`, `/api/v1/history/*`, `/api/v1/ohlcv/*`, `/api/v1/compare/*`	**30 RPS** per-IP	`rl:groupA:{ip}`
B	`/api/v1/orderbook/*`, `/api/v1/spread/*`, `/api/v1/funding/*`, `/api/v1/funding-rate/*`, `/api/v1/slippage/*`, `/api/v1/liquidity/*`, `/api/v1/efficiency/*`, `/api/v1/rsi/*`	**5 RPS** per-IP	`rl:groupB:{ip}`
C	`/api/v1/whale-alerts`, `/api/v1/wallet/*`, `/api/v1/news`, `/api/v1/ico-calendar`, `/api/v1/etf-flows`, `/api/v1/fees`, `/api/v1/all-in-cost/*`	**2 RPS** per-IP	`rl:groupC:{ip}`

Her grup için ayrı `RateLimiter` instance'ı oluşturulur: `NewRateLimiter(redisAddr, name, limit)`. Redis key'i `"rl:" + name + ":" + ip` formatındadır — Grup A'yı tüketen bir IP, Grup B veya C sayaçlarını etkilemez.

Özellik	Detay
Algoritma	Sliding window (Redis ZSET)
Kapsam	IP bazlı, distributed (Redis), grup izolasyonlu
IP tespiti	Önce `X-Forwarded-For`, yoksa `RemoteAddr`
Atomicity	Lua script (`EVALSHA`) — ZREMRANGEBYSCORE + ZCARD + ZADD tek transaction'da
TTL	Her key `PEXPIRE` ile otomatik temizlenir
Redis down	Degraded mode — per-IP in-memory token bucket'a düşer, hata loglanır
Limit aşımı	`429 Too Many Requests` + `{"error":"too many requests"}`

Neden Redis? Her pod kendi sayacını tutsa N pod × limit kadar istek geçebilirdi; gateway restart edilince kota da sıfırlanırdı. Redis ile tüm instance'lar ortak sayacı görür ve kota pod yaşam döngüsünden bağımsız kalır.

Neden Lua script? Pipeline ile yapılan ZCARD→ZADD arasında başka bir request girebilir (TOCTOU). Lua script Redis'te single-threaded execute edildiğinden external lock gerekmez.

3.5 Circuit Breaker (`internal/middleware/circuitbreaker.go`)

Her downstream servis için bağımsız bir circuit breaker instance'ı vardır (`market-data` ve `exchange`). Threshold değerleri hardcoded değil; servis başlatılırken environment variable'dan okunur.

Konfigürasyon

Her servis için prefix `<SERVİS_ADI>_CB_*` formatındadır (`-` → `_` dönüşümü uygulanır):

Env Variable	Varsayılan	Açıklama
`MARKET_DATA_CB_FAILURE_THRESHOLD`	`5`	Kaç 5xx sonra Open'a geçilir
`MARKET_DATA_CB_SUCCESS_THRESHOLD`	`2`	Half-Open'dan Closed için kaç başarı gerekir
`MARKET_DATA_CB_TIMEOUT`	`30s`	Open → Half-Open bekleme süresi
`EXCHANGE_CB_FAILURE_THRESHOLD`	`5`	—
`EXCHANGE_CB_SUCCESS_THRESHOLD`	`2`	—
`EXCHANGE_CB_TIMEOUT`	`30s`	—

`getEnvDuration` Go'nun standart `time.ParseDuration` formatını kabul eder: `30s`, `1m`, `60s`.

Durum Makinesi

```
        FailureThreshold hata
Closed ──────────────────────► Open
  ▲                              │
  │          Timeout             │
```

```
    │    ◄──────────────────── │
    │                       Half-Open
    │   SuccessThreshold başarı │
    └──────────────────────────┘
```

Durum	Açıklama
`Closed`	Normal çalışma — istekler geçer
`Open`	Servis çöktü — tüm istekler direkt 503 döner
`Half-Open`	Test modu — istek geçirilir; `SuccessThreshold` kadar başarı gelince `Closed`'a döner

Hata tespiti: Downstream servisin `5xx` HTTP status kodu dönmesi hata sayılır. `4xx` hatalar (client hataları) sayılmaz.

Open durumunda response:

```
{
  "error": "service temporarily unavailable",
  "service": "market-data"
}
```

Tüm durum geçişleri `sync.Mutex` ile thread-safe'tir.

3.6 Prometheus Metrikleri (`internal/middleware/metrics.go`)

Her HTTP isteği için Prometheus sayaçları ve histogramları günceller.

Metrik	Tip	Açıklama
`http_requests_total`	Counter	method + path + status etiketleriyle toplam istek sayısı
`http_request_duration_seconds`	Histogram	İstek süresi dağılımı
`rate_limit_hits_total`	Counter	Rate limiter tarafından reddedilen istek sayısı
`circuit_breaker_open`	Gauge	CB açıksa 1, kapalıysa 0 (servis bazlı)

`GET /metrics` endpoint'i Prometheus scraper'ına bu verileri sunar.

3.7 Request Logger (`internal/middleware/logging.go`)

Her isteği şu formatta loglar:

```
[2026-04-06 17:00:00] GET /api/v1/quotes/BTCUSDT | status=200 | latency=12ms |
```

```
ip=127.0.0.1:54321 | user=abc123 | request_id=a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

`X-User-ID` header'ı varsa kullanıcı ID'si, yoksa `-` yazar. `request_id` context'ten okunur; `RequestID` middleware zincirde önce geldiğinden her zaman mevcuttur.

3.8 Correlation ID (`internal/middleware/requestid.go`)

Her request'e benzersiz bir UUID atar. Distributed tracing'in temelini oluşturur.

Adım	Açıklama
Header kontrolü	Gelen request'te `X-Request-ID` varsa onu kullanır (client veya upstream gateway'den gelebilir)
UUID üretimi	Header yoksa `crypto/rand` ile UUID v4 üretir — harici bağımlılık gerektirmez
Context	ID, `RequestIDKey` ile request context'ine yazılır — tüm downstream handler'lar okuyabilir
Response header	`X-Request-ID` response'a da eklenir — client hangi ID ile takip edeceğini bilir
Downstream iletimi	`proxy.go` context'ten okuyarak `X-Request-ID` header'ını downstream servise taşır

---

3.9 Health Aggregation (`internal/handler/health.go`)

`GET /health` endpoint'i downstream servislerin `/health` endpoint'lerini kontrol eder ve birleşik durum döner.

Tüm servis kontrolleri `sync.WaitGroup` ile **paralel** olarak çalışır; her biri ayrı bir goroutine'de başlatılır. Tüm kontroller için `context.WithTimeout` ile **global 3 saniyelik deadline** uygulanır — kaç servis olursa olsun toplam bekleme süresi 3s'yi geçemez. Eş zamanlı map yazmaları `sync.Mutex` ile korunur.

Downstream servisler ngrok üzerinden çalıştığından her isteğe `ngrok-skip-browser-warning: true` header'ı eklenir. Herhangi bir servis `down` ise:

- `status` alanı `"degraded"` olur
- HTTP status `503 Service Unavailable` döner

Özellik	Detay
Paralellik	`sync.WaitGroup` + goroutine (servis başına)
Timeout	`context.WithTimeout(r.Context(), 3s)` — global deadline
Race koruması	`sync.Mutex` ile map yazmaları serialize edilir
Context iletimi	`http.NewRequestWithContext` ile context downstream'e taşınır

```
{
  "status": "ok",
  "services": {
```

```
      "market-data": { "status": "ok", "latency": "45ms" },
      "exchange":    { "status": "down", "error": "connection refused" }
    }
  }
```

3.10 WebSocket Proxy (`internal/handler/websocket.go`)

`github.com/gorilla/websocket` ile implement edilmiştir. İki yönlü (bidirectional) tünel açar.

```
Client ─── ws/wss ──► api-gateway ─── ws/wss ──► Upstream
                ◄──────────────────────────────────────
```

Bağlantı sırası:

1. Önce upstream'e bağlanılır — başarısız olursa client'a HTTP 502 dönülür
2. Ardından client HTTP→WebSocket upgrade edilir
3. İki goroutine (her yön için) `errc` channel üzerinden senkronize çalışır — biri kapanınca diğeri de durur

Scheme dönüşümü: HTTP target URL'leri otomatik olarak WebSocket scheme'ine çevrilir:

- `http://` → `ws://`
- `https://` → `wss://`

Header iletimi: Yalnızca `Authorization` ve `X-` prefix'li header'lar upstream'e iletilir. JWT middleware tarafından set edilen `X-User-ID` bu sayede downstream'e ulaşır.

Token query param dönüşümü: Eğer `Authorization` header yoksa `?token=<jwt>` query parametresi `Authorization: Bearer <jwt>` header'ına dönüştürülerek upstream'e iletilir. Bu sayede WebSocket bağlantılarında tarayıcı kısıtlaması aşılır.

TLS: `InsecureSkipVerify: true` — ngrok sertifikalarını doğrulamadan geçer. Production'da kapatılabilir.

Tampon boyutu: Her yönde 32 KB (`gorilla/websocket` upgrader buffer'ı) — orderbook gibi büyük mesajlar için yeterli.

Handshake timeout: Upstream bağlantısı için 10 saniye.

> `/ws/positions/*` JWT korumalıdır — WebSocket handshake'ten önce token doğrulanır.

---

4. Route Tablosu

Gateway Sistem Endpoint'leri

Method	Path	Auth	Açıklama
`GET`	`/health`	—	Downstream servis durumu aggregation
`GET`	`/swagger/*`	—	Swagger UI

Method	Path	Auth	Açıklama
GET	/swagger/doc.yaml	—	OpenAPI YAML tanım dosyası
GET	/metrics	—	Prometheus metrik endpoint'i

market-data — Public (JWT Gerektirmez)

Method	Path	Rate Limit Grubu	Circuit Breaker
ANY	/api/v1/quotes/*	A — 30 RPS	market-data CB
ANY	/api/v1/history/*	A — 30 RPS	market-data CB
ANY	/api/v1/ohlcv/*	A — 30 RPS	market-data CB
ANY	/api/v1/compare/*	A — 30 RPS	market-data CB
ANY	/api/v1/orderbook/*	B — 5 RPS	market-data CB
ANY	/api/v1/spread/*	B — 5 RPS	market-data CB
ANY	/api/v1/funding/*	B — 5 RPS	market-data CB
ANY	/api/v1/funding-rate/*	B — 5 RPS	market-data CB
ANY	/api/v1/slippage/*	B — 5 RPS	market-data CB
ANY	/api/v1/liquidity/*	B — 5 RPS	market-data CB
ANY	/api/v1/efficiency/*	B — 5 RPS	market-data CB
ANY	/api/v1/rsi/*	B — 5 RPS	market-data CB
ANY	/api/v1/whale-alerts	C — 2 RPS	market-data CB
ANY	/api/v1/wallet/*	C — 2 RPS	market-data CB
ANY	/api/v1/news	C — 2 RPS	market-data CB
ANY	/api/v1/ico-calendar	C — 2 RPS	market-data CB
ANY	/api/v1/etf-flows	C — 2 RPS	market-data CB
ANY	/api/v1/fees	C — 2 RPS	market-data CB
ANY	/api/v1/all-in-cost/*	C — 2 RPS	market-data CB
WS	/ws	—	—
WS	/ws/quotes/*	—	—
WS	/ws/orderbook	—	—

exchange-service — Public (JWT Gerektirmez)

Method	Path	Circuit Breaker
ANY	/api/v1/auth/*	exchange CB

exchange-service — Protected (JWT Zorunlu)

Method	Path	Circuit Breaker
ANY	/positions/*	exchange CB
ANY	/api/v1/portfolio/*	exchange CB
ANY	/api/v1/pnl/*	exchange CB
ANY	/api/v1/trades/*	exchange CB
ANY	/api/v1/orders	exchange CB
ANY	/api/v1/orders/*	exchange CB
ANY	/api/v1/apikeys	exchange CB
ANY	/api/v1/apikeys/*	exchange CB
ANY	/api/v1/alerts	exchange CB
ANY	/api/v1/alerts/*	exchange CB
ANY	/api/v1/dca/*	exchange CB
ANY	/api/v1/risk/*	exchange CB
ANY	/api/v1/position/*	exchange CB
ANY	/api/v1/users/*	exchange CB
ANY	/api/v1/performance	exchange CB
ANY	/api/v1/staking	exchange CB
ANY	/api/v1/staking/*	exchange CB
GET	/api/v1/stacks	exchange CB
WS	/ws/positions/*	—
WS	/api/v1/ws	—
WS	/api/v1/ws/*	—

---

5. Konfigürasyon

Değişken	Varsayılan	Açıklama
PORT	9000	Gateway'in dinlediği port
ALLOWED_ORIGIN	http://localhost:3000	CORS için izin verilen frontend origin — credentials:

Değişken	Varsayılan	Açıklama
		`include` ile çalışmak için spesifik domain zorunlu; production'da set edilmeli
`JWT_SECRET`	`"default-secret-change-in-production"`	JWT doğrulama anahtarı — production'da mutlaka set edilmeli
`MARKET_DATA_URL`	`<market-data-ngrok-url>`	market-data servisinin adresi
`EXCHANGE_URL`	`<exchange-ngrok-url>`	exchange-service'in adresi
`REDIS_URL`	`redis:6379`	Redis bağlantı adresi (Docker service name)
`RATE_LIMIT_GROUP_A`	`30`	Grup A rate limit — quotes, history, ohlcv, compare (RPS per-IP)
`RATE_LIMIT_GROUP_B`	`5`	Grup B rate limit — orderbook, spread, funding, slippage, vb. (RPS per-IP)
`RATE_LIMIT_GROUP_C`	`2`	Grup C rate limit — news, whale-alerts, wallet, ico-calendar, vb. (RPS per-IP)
`TIMEOUT_MARKET_DATA`	`5s`	market-data route'ları için upstream timeout (Grup A, B, C)
`TIMEOUT_EXCHANGE`	`10s`	exchange route'ları için upstream timeout (auth + protected)
`MARKET_DATA_CB_FAILURE_THRESHOLD`	`5`	market-data CB — kaç 5xx sonra Open
`MARKET_DATA_CB_SUCCESS_THRESHOLD`	`2`	market-data CB — Half-Open'dan Closed için kaç başarı
`MARKET_DATA_CB_TIMEOUT`	`30s`	market-data CB — Open → Half-Open bekleme süresi
`EXCHANGE_CB_FAILURE_THRESHOLD`	`5`	exchange CB — kaç 5xx sonra Open
`EXCHANGE_CB_SUCCESS_THRESHOLD`	`2`	exchange CB — Half-Open'dan Closed için kaç başarı

Değişken	Varsayılan	Açıklama
`EXCHANGE_CB_TIMEOUT`	`30s`	exchange CB — Open → Half-Open bekleme süresi

> `JWT_SECRET` set edilmezse `auth.go` dev fallback secret kullanır — token doğrulama çalışır ama güvensizdir. Production'da bu değişken zorunludur.

---

6. Dosya Yapısı

```
api-gateway/
├── cmd/
│   └── main.go                    # Uygulama giriş noktası — router kurulumu,
middleware zinciri, route tanımları
├── config/
│   └── config.go                  # Environment variable okuma ve varsayılan
değerler
├── docs/
│   └── swagger.yaml               # OpenAPI 3.0 — tüm endpoint tanımları,
parametreler, auth şeması
├── internal/
│   ├── handler/
│   │   ├── health.go              # GET /health — downstream servis durumu
aggregation
│   │   ├── proxy.go               # HTTP reverse proxy (httputil.ReverseProxy
wrapper)
│   │   └── websocket.go           # WebSocket bidirectional tünel
│   └── middleware/
│       ├── auth.go                # JWT doğrulama — token parse, X-User-ID
iletimi
│       ├── circuitbreaker.go      # Circuit breaker state machine (Closed / Open
/ Half-Open)
│       ├── circuitbreaker_test.go # Circuit breaker unit testleri
│       ├── metrics.go             # Prometheus metrikleri — istek sayısı, süre,
rate limit, CB durumu
│       ├── logging.go             # RequestLogger — method, path, status,
latency, IP, user, request_id loglama
│       ├── requestid.go           # Correlation ID — UUID üretimi, context yazma,
downstream iletimi
│       ├── ratelimit.go           # IP bazlı Redis sliding window rate limiter
│       └── timeout.go             # Per-route upstream timeout — context
deadline, 504 on exceed
├── Dockerfile                     # Multi-stage build — Go binary derleme +
minimal runtime image
├── docker-compose.yml             # Gateway + Redis servislerini environment
variable'larla ayağa kaldırır
├── go.mod                         # Go modül tanımı ve bağımlılık listesi
└── go.sum                         # Bağımlılık checksum'ları
```

---

7. Test Sonuçları

Aşağıdaki testler ngrok üzerinden gerçek ortamda gerçekleştirilmiştir.

7.1 Health Aggregation

```
curl https://<gateway-ngrok>/health
# → {
#     "status": "ok",
#     "services": {
#       "market-data": { "status": "ok", "latency": "38ms" },
#       "exchange":    { "status": "ok", "latency": "52ms" }
#     }
#   }
```

✓ 3 servis de "ok" döndü — servisler arası bağlantı doğrulandı

7.2 JWT Koruması

```
# Token olmadan
curl https://<gateway-ngrok>/positions/1
# → {"error":"unauthorized"}  HTTP 401

# Geçerli token ile
curl -H "Authorization: Bearer <token>" https://<gateway-ngrok>/positions/1
# → pozisyon listesi  HTTP 200
```

✓ Korumasız istek reddedildi, geçerli token ile geçti

7.3 Rate Limiting

```
# Grup A — 31. istekte (quotes)
curl https://<gateway-ngrok>/api/v1/quotes/BTCUSDT  # x31
# → {"error":"too many requests"}  HTTP 429

# Grup C — 3. istekte (whale-alerts)
curl https://<gateway-ngrok>/api/v1/whale-alerts  # x3
# → {"error":"too many requests"}  HTTP 429

# Grup izolasyonu — Grup C doluyken Grup A etkilenmez
# → Grup A istekleri 200 dönmeye devam eder
```

✓ Her grubun limiti bağımsız çalışıyor, gruplar arası sızma yok

7.4 Circuit Breaker

```
# exchange-service kapalıyken
curl -H "Authorization: Bearer <token>" https://<gateway-ngrok>/positions/1
# → {
#     "error": "service temporarily unavailable",
#     "service": "exchange"
#   }  HTTP 503
```

✓ 5 art arda hata sonrası devre açıldı, 30 saniye sonra otomatik kapandı

---

8. Swagger UI Kullanım Kılavuzu

Gateway çalışırken Terminale go run ./cmd/main.go `http://localhost:9000/swagger/index.html` adresini tarayıcıda aç.

8.1 Public Endpoint Test Etmek

1. Listeden bir endpoint'e tıkla (örn. `GET /api/v1/quotes/{symbol}`)
2. **Try it out** butonuna bas
3. Parametre kutularını doldur (örn. `symbol` = `BTCUSDT`)
4. **Execute** bas — gerçek API yanıtı altta görünür

8.2 JWT Gerektiren Endpoint Test Etmek

1. Önce token al:
   - `POST /api/v1/auth/login` → Try it out → email/password gir → Execute
   - Response'dan `token` değerini kopyala
2. Sağ üstteki **Authorize** butonuna tıkla
   - Value kutusuna token'ı yapıştır → **Authorize** → Close
3. Kilit ikonu olan endpoint'lere artık otomatik JWT ile istek atılır

8.3 API Tanım Dosyası

`docs/swagger.yaml` — tüm endpoint tanımları burada. Yeni endpoint eklendiğinde bu dosyaya da ilgili path eklenmeli.

---

9. Bilinen Kısıtlamalar ve Sonraki Adımlar

#	Kısıtlama	Durum	Planlanan Çözüm
1	WebSocket proxy'de circuit breaker yok	Açık	WS bağlantıları CB'den geçirilmeli
2	JWT token revocation yok	Açık	Token invalidation / blacklist mekanizması

#	Kısıtlama	Durum	Planlanan Çözüm
3	Distributed tracing yok	Kısmi ✓	X-Request-ID correlation ID eklendi; Jaeger / OpenTelemetry entegrasyonu sonraki adım

---

Tresaurio Platform — api-gateway v0.9.1